

模型验证报告

Group 7

2025 年 11 月 17 日

1 引言

本报告将从以下几个维度，对 Group 8 的完整代码和建模流程进行验证：

- 代码是否能顺利运行，结果是否具有可复现性（Replicability）；
- 数据预处理与特征工程是否合理、是否存在隐含的错误或不一致；
- 训练集、验证集、测试集的划分是否规范，是否存在数据泄露（Data Leakage）；
- 模型性能报告是否可信，是否表现出过拟合、欠拟合或不稳定；
- 从模型风险管理角度提出若干改进建议。

2 被验证项目概述

对方小组的项目包含两个核心预测任务：

1. 房价（Price）预测任务；
2. 租金（Rent）预测任务。

在每一个任务中，整体流程基本一致，主要包括：

- **数据加载：**使用 `load_data` 函数，兼容 GBK 和 UTF-8 编码，读取训练与测试 CSV 文件。
- **特征工程：**通过自定义函数解析和构造大量结构化特征，例如：
 - 户型解析：将“3 室 2 厅 1 厨 2 卫”等字段拆解为 Room / Hall / Kitchen / Bath 等数值特征；
 - 楼层解析：将“中楼层 (共 23 层)”或“4/6 层”解析为楼层等级、所在层数和总楼层数；

- 面积解析：从“86.94 ㎡”等字符串中抽取数值面积；
 - 梯户比例解析：将“1 梯 2 户”“一梯二户”等文本统一解析为电梯数量与户数；
 - 年代与房龄：解析“建筑年代”（如“2002-2006 年”），结合“交易时间”得到房龄（HouseAge）；
 - 绿化率、容积率、物业费、停车位、停车费等字段由字符串清洗为数值；
 - 基于经纬度和城市字段进行地理聚类（`create_geo_clusters`），构造“City_城市_聚类编号”这样的地理类别特征；
 - 构造交互特征，如总房间数（TotalRooms）、套内面积占比（InnerAreaRatio）、楼层占比（FloorRatio）等；
 - Price 与 Rent 两个任务之间还利用板块（小区、板块）维度的平均租金/售价做交叉特征。
- **冗余与泄露字段处理：**显式删除“核心卖点”“房屋优势”等可能携带目标信息但在真实预测场景中无法使用的字段，同时去除部分坐标与冗余年份字段。
 - **样本划分：**使用 `train_test_split(X, y, test_size=0.2, random_state=111)` 将训练数据拆分为训练集与测试集。
 - **预处理：**
 - 对数值特征：中位数填补 + 标准化（线性模型）或中位数填补（LGBM）；
 - 对类别特征：众数填补 + One-Hot 编码（线性模型）或 Ordinal 编码（LGBM）；
 - 使用 `ColumnTransformer` 将数值与类别预处理组合，构建 Pipeline。
 - **模型训练与调参：**引入多种模型对比，包括 OLS、Lasso、Ridge、ElasticNet，以及 LightGBM，并使用 6 折交叉验证（`KFold(n_splits=6)`）以及自定义的 MAE 评分函数（在原始价格尺度上评估）。
 - **结果汇总与提交：**输出包含 In-sample MAE、Out-of-sample MAE、6-fold CV MAE 的表格，并基于每个模型生成 Kaggle 提交文件（`submission_combined_模型名.xlsx`）。

从功能覆盖和工程结构来看，对方小组的项目是一个相对完整的“端到端”机器学习建模流程，包含从数据清洗、特征工程到模型比较与线上提交的完整链路。

3 代码可执行性与结果可复现性

结论：在给定的数据文件和目录结构下，对方小组代码可以顺利执行，性能指标与其报告中的结果基本一致，具有较好的可复现性。

4 数据处理与特征工程评估

本节我们从数据清洗、特征构造与异常值处理三个角度，对项目的数据处理部分进行验证。

4.1 数据清洗与解析

从业务含义来看，对房屋相关字段进行结构化解析是非常有价值的。对方小组在这方面做了大量工作，包括：

- 将复杂的中文文本字段（户型、楼层、面积、梯户比例、停车信息、建筑年代等）转化为可被模型利用的数值或有序类别特征；
- 用日期差计算房龄（HouseAge），体现了对时间维度的重视；
- 对“配套设施”字段按照“项目数量”计数，得到配套设施的丰富程度；
- 在城市内部基于经纬度进行 KMeans 聚类，得到地理位置层面的类别特征。

这些特征工程操作与房地产定价逻辑是高度契合的：房龄、楼层、面积、户型结构、地理位置等本就是影响价格与租金的关键因素。因此，从“变量选取是否有经济学和业务逻辑支撑”的角度看，特征工程整体是合理而且用心的。

4.2 泄露列与冗余列的处理

对方小组显式丢弃了多列可能导致目标泄露或在真实预测中不可用的字段，如“核心卖点”“房屋优势”“户型介绍”“交通出行”以及部分产权信息、物业公司、开发商等字段，同时也去掉了冗余坐标与“年份”字段。这体现了对“信息可获得性”与“潜在泄露风险”的基本意识，是模型风险管理中的一个加分项。

4.3 异常值处理

在 Price 与 Rent 任务中，对方小组均采用了 IQR（四分位距）规则对价格和面积进行异常值过滤。例如：

- 价格仅保留在 $[Q_1 - 1.5 \cdot IQR, Q_3 + 1.5 \cdot IQR]$ 范围内的样本；
- 面积方面，Price 任务中要求建筑面积在 10–1000 之间，Rent 任务中则为 5–500。

这一做法有助于剔除极端异常房源，对模型稳健性是有利的。但在技术细节上，需要注意以下两点：

- 异常值过滤应确保使用的标签与过滤后的特征集合在索引上完全对齐，避免出现 mask 长度不一致或标签错位的风险；

- 基于价格的 IQR 统计量，通常应仅基于训练集计算，而不应混入将来用作验证/测试的样本。

整体来看，异常值处理的方向是正确的，但细节仍有提升空间（见第 6 节建议）。

5 样本划分与数据泄露检查

模型验证中最关键的一环，是检查是否存在**数据泄露 (Data Leakage)**：即验证集 / 测试集的信息在某种形式上“提前”被用到了模型训练或特征工程中，导致评估结果过于乐观。

5.1 基础划分：train__test__split

对方小组使用

```
train_test_split(X, y, test_size=0.2, random_state=111)
```

将处理后的训练数据集划分为训练集和测试集，在形式上是正确的：使用固定随机种子，保证结果可复现，且 8:2 的比例在工业实践中也十分常见。

5.2 预处理与特征工程的作用范围

在 Price 和 Rent 任务中，原始训练集与测试集 (Kaggle test) 被纵向拼接之后，统一做了特征工程和部分预处理操作，再根据 source 字段拆分为训练部分与测试部分。这种“先合并，再处理，最后拆分”的写法在工程上相对简单，但在模型验证视角下存在以下隐患：

- 若在“合并后的整体数据”上进行编码器（如 One-Hot / OrdinalEncoder）的 fit，那么**测试集**的类别信息会提前出现在训练阶段，导致轻微的数据泄露；
- 在 LightGBM 部分，preprocessor_lgbm 对完整的 X 再次使用 fit_transform，会让交叉验证对模型性能的估计偏乐观，而真正的 out-of-sample 测试表现较差，这一点也体现在 LGBM 的结果上——交叉验证 MAE 显著优于测试 MAE。

从严格的模型验证角度，应当遵循以下原则：

- 所有带有“学习性质”的变换（标准化、编码、聚类等）必须只在训练集上 fit，再对验证集 / 测试集 transform；
- 尽量避免将“训练集 + 测试集”完整拼在一起再做 fit_transform 的写法。

5.3 地理聚类与交叉特征

在地理聚类部分，对方小组使用 `create_geo_clusters` 函数按城市进行 KMeans 聚类，并将聚类标签与城市名称拼接为新类别特征。从业务逻辑看，这个特征是有意义的，但在实现上也需要注意：

- 当前代码中 `create_geo_clusters` 在 Price 与 Rent 部分各定义了一次，且是针对“合并后的数据”做聚类；
- 若聚类在“训练 + 测试”或“训练 + 验证 + 测试”整体上完成，则测试样本的位置会影响聚类结果，从而构成信息泄露风险；
- 在理想实现中，应当在训练集上为每个城市分别 `fit` 一个 KMeans，再将同一个聚类模型应用到验证集和测试集，以保证聚类结构与真实预测场景一致。

总的来说，对方小组在样本划分上遵循了基本规范，但在“预处理 / 聚类是否仅使用训练集拟合”这一点上仍存在改进空间。

6 模型验证结论与改进建议

结合以上分析，我们认为对方小组的项目在工程完备性和特征工程丰富度方面表现优秀，但在“严格意义上的模型验证”维度仍存在改进空间。下面给出若干条具有可操作性的改进建议。

建议 1：避免在“训练 + 测试”合并后进行预处理的 `fit`，严格在训练集上拟合变换

当前代码中，训练数据和 Kaggle 测试数据在部分步骤被合并成一个整体 DataFrame，再对其进行部分特征工程和预处理。在 LightGBM 部分，还存在对完整 X 调用 `fit_transform` 的情况。这样会导致：

- 测试集类别水平（category levels）参与到了编码器的拟合；
- 交叉验证时的数据划分与实际 `train_test_split` 的评估不完全一致，出现“CV 很好、Test 很差”的现象。

更规范的做法是：

- 先在训练集上完成 `fit`（包括标准化、编码、聚类步骤）；
- 再将同样的变换以 `transform` 方式应用到验证集和测试集；
- 避免在全量数据上做 `fit_transform`。

建议 2: `create_geo_clusters()` 函数应统一定义并避免对“整体数据”聚类

`create_geo_clusters()` 函数在 `price` 和 `rent` 部分重复定义，可删去后者；该函数基于地理信息进行聚类，但在处理时是对整个 `train` 数据集进行聚类，没有对测试集和验证集进行分别聚类，这样虽然对 `test` 数据集的预测不会造成影响，但是会影响验证集的检验效果，不利于模型调优，建议在 `split` 测试集和验证集后对两个数据集分别进行聚类操作。

建议 3: 拆分训练集和验证集之后，再分别完成聚类与特征工程，以提高验证效果

当前代码中将训练集和测试集合并再做预处理，这会导致测试集信息泄漏，应当先使用 `train_test_split` 将训练数据随机分为训练集和验证集，在训练集上完成数据处理、特征工程操作后再将相同变换应用到验证集；建议优化代码结构，代码在一些模块的定义上有多处重复（如 `create_geo_clusters`），应将公共功能抽象为函数或模块，只定义一次并复用。更严格的做法是：

- 先将原始训练数据拆分为训练子集和验证子集；
- 在训练子集上完成地理聚类与其他“带学习性质”的特征工程操作；
- 使用相同的规则或模型将这些变换应用到验证子集；
- 这样，验证集的评估结果才能真实反映“新数据”上的模型表现。

建议 4: 针对城市层级字段缺失和极端情况，提升特征工程的健壮性

在城市层级字段缺失的场景下存在以下隐患，会导致缺失值填充策略失效甚至模型输入维度不一致：

- 某城市数据里根本没有‘板块’/‘区域’/‘聚类 _ 城市’这三列，但代码仍把它们写死到 `if col in [...]` 列表里；结果：`ColumnTransformer` 会得到空列表，运行期报 `ValueError: No columns were specified`；或者字段全为 `NaN`，后续 `SimpleImputer(strategy='most_frequent')` 填的全是‘Missing’，一列唯一值 `=1`，`OneHot` 后生成全 0 稀疏矩阵，白白吃掉内存。
- 缺失值填充顺序反了：先 `fillna('Missing')` 再 `Imputer('most_frequent')` 导致众数永远是‘Missing’，等于没填补。
- 异常值过滤只用 `train_processed` 做面积 `mask`，却用全量 `y_target` 计算 `IQR`。虽然时间无序，但面积 `mask` 的列已经只来自训练集，所以 `IQR` 也应与训练集标签对应，否则 `mask` 长度不一致。

建议在这些边缘情形下：

- 显式检查 ColumnTransformer 中每一组列是否为空，若为空应跳过该 transformer 或给出合理的默认行为；
- 将“填 Missing”与“基于众数填补”的逻辑进行梳理，避免重复或逻辑冲突；
- 确保异常值过滤操作始终在同一个样本集合上计算统计量与应用 mask，使索引严格对齐。

建议 5：扩展正则化线性模型与 LightGBM 的超参数搜索空间

目前 Lasso / Ridge / ElasticNet 以及 LightGBM 的超参数网格相对较小，例如：

- Lasso 的 `alpha` 仅取 `[0.001, 0.01, 0.1]`；
- Ridge 的 `alpha` 仅取 `[0.1, 1.0, 10.0]`；
- LightGBM 只对 `n_estimators` 和 `learning_rate` 做了简单搜索。

在模型验证的视角下，过于狭窄的网格可能导致“调参不充分”的风险，使得模型未能达到其应有的最优表现。建议：

- 对线性正则化模型扩大 `alpha` 范围，如 `[10-4, 10-3, 10-2, 10-1, 1, 10, 100]`；
- 对 LightGBM 增加对 `max_depth`、`num_leaves`、`min_data_in_leaf`、`feature_fraction` 等关键超参数的调节；
- 在可能的情况下，结合随机搜索或贝叶斯优化等方法提高调参效率。

建议 6：高基数类别变量可考虑做频数截断或目标编码，以提高模型稳定性

在当前实现中，城市、板块、区县等类别特征直接通过 One-Hot 或 Ordinal 编码喂给模型，这在高基数场景下会导致：

- 特征维度极其庞大，线性模型在估计时容易出现病态矩阵和多重共线性；
- 对 LightGBM 而言，会产生大量类别 bins，引发 warning，影响训练效率和稳定性。

建议：

- 对类别频数较低的长尾类别进行合并（如截断到前若干热门类别，其余归类为“Other”）；
- 在严格的交叉验证框架下尝试目标编码（Target Encoding），但需防止再次引入泄露；
- 对部分高基数变量，可以考虑通过降维或聚类先进行压缩。

7 总结

综上所述，我们认为 Group 8 的机器学习项目在以下几个方面表现突出：

- 完整实现了房价与租金双任务的建模闭环；
- 特征工程深入、充分考虑了房地产业务逻辑；
- 形式上引入了多种模型并进行了交叉验证与性能对比；
- 代码整体可运行、结果可复现。

与此同时，从**模型验证与模型风险管理**的角度看，项目仍存在以下需要改进之处：

- 预处理与聚类在“训练 + 测试”整体数据上的 `fit` 存在信息泄露隐患；
- LightGBM 的交叉验证结果显著优于真实测试表现，提示验证流程本身可能存在偏差；
- 对城市层级字段缺失、极端案例以及异常值处理的细节仍有提升空间；
- 超参数搜索相对粗略，高基数类别变量尚未进行针对性的降维或编码优化。

本报告给出的若干建议，既是对对方小组项目的改进方向，也是对我们自身“如何成为一个批判性思考者 (critical thinker)”的一次训练——不仅要“把模型做对”，更要“把模型验证做对”。